

Ejercicio en clase – 30 de septiembre

MPI_Gather() y MPI_Reduce()

Sebastian Estrada - 21405
Maria Jose Yee - 231193

Caso 1. Recolección de temperaturas con MPI_Gather()

a) Parámetros de MPI_Gather() (programa original Ejercicio1a.c)

Parámetro	Valor utilizado	Explicación
sendbuf	&temperatura	Es la dirección de la variable local de cada proceso que contiene la temperatura.
sendcount	1	Cantidad de elementos que envía en cada proceso.
sendtype	MPI_FLOAT	Es el tipo de dato de los elementos que se envían en este caso es un float.
recvbuf	temperaturas	Lista de 4 floats donde el proceso root almacena los valores que se obtienen.
recvcount	1	Es la cantidad de elementos que el root recibe <i>de cada</i> proceso.
recvtype	MPI_FLOAT	Tipo de dato de los elementos recibidos.
root	0	Rank del proceso que recibe y junta los datos.
communicator	MPI_COMM_WORLD	Grupo de procesos que participan en la comunicación (todos los procesos lanzados).

b) Modificación (Ejercicio1a_modificado.c)

Cada proceso guarda dos mediciones dentro de un arreglo local que se llama float temperatura[2]. Por esto, el arreglo del proceso root ahora tiene 8 posiciones, es decir, básicamente almacena las 8 temperaturas obtenidas. En la llamada a MPI_Gather() se cambió el sendbuf para usar la temperatura. Al ser un arreglo, ya no es necesario colocar "&". También se cambiaron sendcount y recvcount de 1 a 2, debido a que cada proceso envía dos valores en lugar de uno.

El proceso root obtiene los dos valores correspondientes a cada proceso `i` desde las posiciones `temperaturas[2*i]` y `temperaturas[2*i+1]`.

```
MPI_Gather(temperatura, 2, MPI_FLOAT, temperaturas, 2, MPI_FLOAT, 0, MPI_COMM_WORLD)
```

Evidencia de compilación y ejecución

Original:

```
sestradat@compu-sebas:~/Escritorio/u/semestre/raro 1 /Paralela/30-de-sep$ ./ejecutar_todo.sh -p
===== Ejerciciola.c =====
$ mpicc Ejerciciola.c -o ejerciciola
$ mpirun -np 4 ./ejerciciola
Proceso 2: temperatura registrada = 23.8 C
Proceso 0: temperatura registrada = 24.5 C

Oficina Central: temperaturas recibidas
Proceso 0: 24.5 C
Proceso 1: 26.1 C
Proceso 1: temperatura registrada = 26.1 C
Proceso 2: 23.8 C
Proceso 3: 27.0 C
Proceso 3: temperatura registrada = 27.0 C
```

Modificado:

```
$ mpicc Ejerciciola_modificado.c -o ejerciciola_mod
$ mpirun -np 4 ./ejerciciola_mod
Proceso 1: temperaturas registradas = 26.1 C, 26.4 C
Proceso 2: temperaturas registradas = 23.8 C, 24.1 C
Proceso 3: temperaturas registradas = 27.0 C, 27.3 C
Proceso 0: temperaturas registradas = 24.5 C, 25.0 C

Oficina Central: temperaturas recibidas
Proceso 0: 24.5 C, 25.0 C
Proceso 1: 26.1 C, 26.4 C
Proceso 2: 23.8 C, 24.1 C
Proceso 3: 27.0 C, 27.3 C
```

Preguntas de análisis

1. ¿Por qué `MPI_Gather()` conserva los resultados individuales de cada proceso?

Es por que recolecta o copia los datos, no los combina, que se envían a cada proceso a una posición distinta del buffer que recibe el root, sin que se modifiquen. Por eso el root termina con un arreglo que contiene cada valor original por separado.

2. ¿Qué relación existe entre la posición de los datos en recvbuf y el rank de cada proceso?

Los datos se ordenan en base al rank del emisor, en donde el rank i guarda a partir de $i \times \text{recvcount}$ del recvbuf. Cuando recvcount es 1, la temperatura del proceso i se guarda en temperaturas[i], en cambio cuando es 2 se guarda en temperaturas[$2i$] y temperaturas[$2i+1$]. Esto hace que el orden no dependa de que un proceso termine primero, como se muestra en la ejecución por que localmente se muestran en desorden pero al imprimir en el root salen ordenados.

3. Si cada proceso enviara dos valores en lugar de uno, ¿qué cambios serían necesarios en los parámetros de MPI_Gather()?

Basicamente sendbuf debe apuntar a un arreglo local de 2 elementos, sendcount y recvcount tiene que ser 2 en vez de 1 y recvbuf tiene que tener capacidad para procesar 2 veces el tamaño de elementos (8 floats con 4 procesos).

Caso 2. Consumo eléctrico con MPI_Reduce()

a) Parámetros de MPI_Reduce() (programa original Ejercicio1b.c)

Parámetro	Valor utilizado	Explicación
sendbuf	&consumo	Es la dirección del consumo local de cada proceso, que es el valor que aporta a la reducción.
recvbuf	&consumo_total	Es la variable donde el root guarda el resultado combinado y que solo se puede acceder desde root.
count	1	Es el número de elementos que aporta cada proceso y que se reducen.
datatype	MPI_INT	Es el tipo de dato de los elementos en este caso es INT.
op	MPI_SUM	Es la operación de que se va a realizar, en este caso es suma
root	0	Rank del proceso que recibe el resultado.
communicator	MPI_COMM_WORLD	Grupo de procesos que participan en la reducción.

b) Modificación (Ejercicio1b_modificado.c)

Se agregaron las variables consumo_maximo y consumo_minimo y dos nuevas llamadas a MPI_Reduce() con las operaciones MPI_MAX y MPI_MIN. El root imprime los tres resultados.

```
MPI_Reduce(&consumo, &consumo_total, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD)
MPI_Reduce(&consumo, &consumo_maximo, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD)
MPI_Reduce(&consumo, &consumo_minimo, 1, MPI_INT, MPI_MIN, 0, MPI_COMM_WORLD)
```

Evidencia de compilación y ejecución

Original:

```
===== Ejercicio1b.c =====
$ mpicc Ejercicio1b.c -o ejercicio1b
$ mpirun -np 4 ./ejercicio1b
Proceso 0: consumo = 150 kWh
Proceso 1: consumo = 120 kWh
Proceso 3: consumo = 100 kWh
Proceso 2: consumo = 180 kWh

Consumo total: 550 kWh

[Enter para continuar al siguiente programa]
```

Modificado:

```
===== Ejercicio1b_modificado.c =====
$ mpicc Ejercicio1b_modificado.c -o ejercicio1b_mod
$ mpirun -np 4 ./ejercicio1b_mod
Proceso 0: consumo = 150 kWh

Consumo total: 550 kWh
Consumo maximo: 180 kWh
Consumo minimo: 100 kWh
Proceso 1: consumo = 120 kWh
Proceso 2: consumo = 180 kWh
Proceso 3: consumo = 100 kWh

[Enter para continuar al siguiente programa]
```

Preguntas de análisis

1. ¿Qué función cumple el parámetro op dentro de MPI_Reduce()?

Permite decir si la operación que se aplica para combinar los valores que se envían a todos los procesos en un único resultado que recibe el root. Tiene que ser una operación asociativa por que permite que MPI combine los valores en cualquier grupo.

2. ¿Qué diferencia existe entre utilizar MPI_SUM, MPI_MAX y MPI_MIN?

Las tres reducen los mismos datos a un solo valor, pero usando una operación distinta:

MPI_SUM suma todos los valores generando un consumo total, MPI_MAX devuelve el mayor valor indicando el consumo mas alto y en que sucursal y MPI_MIN el valor que es menos. Con MAX y MIN el resultado es uno de los valores originales; con SUM es un valor nuevo.

3. ¿Cuál es la diferencia conceptual entre MPI_Gather() y MPI_Reduce()?

Gather basicamente busca recolectar valores y ordenarlos por rank devolviendolos en un tamaño $\text{size} \times \text{count}$, y de esta manera tampoco se pierde informacion.

Reduce basicamente busca combinar utilizando operaciones en las que se puede mantener valores originales o valores nuevos, y el root es quien recibe este resultado.